



TEMA 11

Métodos de Arrays — `forEach`, `map`, `filter`, `find`, `every` e `reduce`

 Conceitos

 12 questões

 `forEach` | `map` | `filter` | `find` | `every` | `reduce`



O que são Métodos de Arrays?

 **Analogia:** Métodos de arrays são como **ferramentas** em uma caixa de ferramentas. Cada ferramenta tem uma função específica: uma para **transformar**, outra para **filtrar**, outra para **encontrar**, etc.

JavaScript possui diversos métodos de array que facilitam a manipulação de dados, tornando o código mais **limpo**, **legível** e **expressivo**. Alguns desses métodos fazem parte do ES6 e outros já existiam em versões anteriores da linguagem.

 **Exemplo do dia a dia:** Imagine que você tem uma lista de números e quer calcular o dobro de cada um. Antes você usaria um loop `for`. Agora, com `map()`, fica muito mais simples e elegante!

✓ Vantagens dos métodos de array:

-  **Código mais curto** e legível
-  **Não alteram o array original diretamente** — métodos como `map()` e `filter()` retornam novos resultados
-  **Encadeáveis** (podem ser combinados)
-  **Expressivos** (o nome diz o que faz)



Resumo dos Métodos

Método	O que faz	Retorna	Exemplo
forEach()	Percorre cada elemento	Nada (undefined)	<pre>array.forEach(item => console.log(item))</pre>
map()	Transforma cada elemento	Novo array (mesmo tamanho)	<pre>array.map(item => item * 2)</pre>
filter()	Filtra elementos que atendem à condição	Novo array (menor ou igual)	<pre>array.filter(item => item > 10)</pre>
find()	Encontra o primeiro elemento que atende à condição	Elemento ou undefined	<pre>array.find(item => item.nome === "Ana")</pre>
every()	Verifica se TODOS atendem à condição	true ou false	<pre>array.every(item => item > 0)</pre>
reduce()	Acumula valores em um único resultado	Valor acumulado	<pre>array.reduce((acc, item) => acc + item, 0)</pre>

⚠ Ordem de aprendizado recomendada:

1. `forEach()` → percorre os elementos de forma simples
2. `map()` → transforma elementos
3. `filter()` → filtra elementos
4. `find()` → encontra um elemento
5. `every()` → verifica todos
6. `reduce()` → acumula valores (mais avançado)

! map() × forEach() — Qual a diferença?

Essa é provavelmente a **confusão número 1** de iniciantes. Entenda a diferença fundamental:

Método	Objetivo	Retorno
forEach()	Executar uma ação para cada elemento	undefined
map()	Transformar cada elemento	Novo array

💡 Exemplo prático:

```
let numeros = [1, 2, 3];

numeros.forEach(numero => {
  console.log(numero * 2); // apenas exibe, não retorna nada
});

// map() → cria um novo array
let dobrados = numeros.map(numero => numero * 2);
console.log(dobrados); // [2, 4, 6] → novo array
```



Exemplos de Código



forEach() - Percorrer Elementos

forEach.js

```
let nomes = ["Ana", "João", "Maria", "Pedro"];

// 1. Exibir cada número no console
console.log("📌 Números:");
numeros.forEach(function(item) {
    console.log(item);
});

// 2. Exibir cada nome com seu índice
console.log("📌 Nomes com índice:");
nomes.forEach(function(nome, indice) {
    console.log("Índice " + indice + ": " + nome);
});

// 3. Usando arrow function (mais moderno)
console.log("📌 Arrow function:");
numeros.forEach((item) => {
    console.log(item * 2);
});
```



map() - Transformar Elementos

map.js

```
// =====
// map() - Transforma cada elemento em um novo array
// =====

let numeros = [1, 2, 3, 4, 5];
let pessoas = [
    { nome: "Ana", idade: 25 },
    { nome: "João", idade: 30 },
    { nome: "Maria", idade: 22 }
];

// 1. Calcular o dobro de cada número
let dobros = numeros.map(function(item) {
    return item * 2;
});
console.log("📌 Dobros:", dobros);

// 2. Extrair apenas os nomes das pessoas
let nomes = pessoas.map(function(pessoa) {
```

📌 filter() - Filtrar Elementos

filter.js

```
// =====  
// filter() - Filtra elementos que atendem à condição  
// =====  
  
let numeros = [5, 12, 3, 18, 7, 20, 9];  
let pessoas = [  
  { nome: "Ana", idade: 25 },  
  { nome: "João", idade: 30 },  
  { nome: "Maria", idade: 22 },  
  { nome: "Pedro", idade: 28 },  
  { nome: "Carla", idade: 17 }  
];  
  
// 1. Filtrar números maiores que 10  
let maiores = numeros.filter(function(item) {  
  return item > 10;  
});  
console.log("📌 Números > 10:", maiores);
```

📌 find() - Encontrar um Elemento

find.js

```
{ nome: "Maria", idade: 22 },  
{ nome: "Pedro", idade: 28 }  
];  
  
// 1. Encontrar o primeiro número maior que 10  
let primeiroMaior = numeros.find(function(item) {  
  return item > 10;  
});  
console.log("📌 Primeiro > 10:", primeiroMaior);  
  
// 2. Encontrar a pessoa com nome "Maria"  
let maria = pessoas.find(function(pessoa) {  
  return pessoa.nome === "Maria";  
});  
console.log("📌 Maria:", maria);  
  
// 3. Tentar encontrar uma pessoa que não existe  
let carlos = pessoas.find((pessoa) => pessoa.nome === "Carlos");  
console.log("📌 Carlos (não existe):", carlos); // undefined
```

📌 every() - Verificar Todos

every.js

```
let numeros = [5, 12, 3, 18, 7];
let idades = [12, 18, 22, 15, 19];

// 1. Verificar se todos os números são positivos
let todosPositivos = numeros.every(function(item) {
    return item > 0;
});
console.log("🚀 Todos são positivos?", todosPositivos);

// 2. Verificar se todas as idades são >= 18
let todosMaiores = idades.every(function(idade) {
    return idade >= 18;
});
console.log("🚀 Todos são maiores de idade?", todosMaiores);

// 3. Verificar se todos os números são pares
let todosPares = numeros.every((item) => item % 2 === 0);
console.log("🚀 Todos são pares?", todosPares);
```

🚀 reduce() - Acumular Valores

reduce.js

```
// =====
// reduce() - Acumula valores em um único resultado
// =====

let numeros = [1, 2, 3, 4, 5];
let pessoas = [
    { nome: "Ana", idade: 25 },
    { nome: "João", idade: 30 },
    { nome: "Maria", idade: 22 }
];

// 1. Somar todos os números
let soma = numeros.reduce(function(accumulator, item) {
    return accumulator + item;
}, 0);
console.log("🚀 Soma:", soma);

// 2. Somar todas as idades
let somaIdades = pessoas.reduce(function(accumulator, pessoa) {
```

💡 **Dica:** `map()` e `filter()` são muito úteis no dia a dia para transformar e filtrar dados, especialmente ao trabalhar com listas de objetos.

Resumo dos Métodos

forEach()

Percorre todos os elementos

```
array.forEach(item => {  
  // ação  
});
```

map()

Transforma cada elemento

```
let novo = array.map(item => item * 2);
```

filter()

Filtra elementos

```
let filtrado = array.filter(item => item > 10);
```

find()

Encontra o primeiro

```
let encontrado = array.find(item => item.nome === "Ana");
```

every()

Verifica todos

```
let resultado = array.every(item => item > 0);
```

reduce()

Acumula valores

```
let soma = array.reduce((acc, item) => acc + item, 0);
```



Exercício de Fixação

Responda as **12 questões** sobre Métodos de Arrays.

Ao final, clique em "**Verificar Respostas**" para corrigir.

1 Qual método é usado para **percorrer** todos os elementos de um array?

☐ map()

☐ filter()

☒ forEach()

☐ find()

2 Qual método **transforma** cada elemento em um novo array?

☐ forEach()

☒ map()

☐ filter()

☐ find()

3 Qual método **filtra** elementos que atendem a uma condição?

☐ map()

☐ forEach()

☒ filter()

☐ reduce()

4 Qual método encontra o **primeiro** elemento que atende a uma condição?

☐ filter()

☒ find()☐ every()☐ map()

5 Qual método verifica se **TODOS** os elementos atendem a uma condição?

☐ some()☒ every()☐ find()☐ filter()

6 Analise o código:

```
let numeros = [1, 2, 3, 4];  
let resultado = numeros.map(function(num) {  
    return num * 2;  
});  
console.log(resultado);
```

Qual será a saída no console?

☐ [1, 2, 3, 4]☒ [2, 4, 6, 8]☐ [2, 4, 6]☐ [1, 2]

7 Analise o código:

```
let idades = [12, 18, 22, 15];  
let maiores = idades.filter(function(idade) {  
    return idade >= 18;  
});
```

```
});  
console.log(maiores);
```

Qual será a saída no console?

☐ [12, 18, 22, 15]

☒ [18, 22]

☐ [12, 15]

☐ [22]

8 Qual método **acumulava** valores em um único resultado?

☐ map()

☐ filter()

☒ reduce()

☐ forEach()

9 O método `map()` modifica o array original?

☐ Sim, modifica o original

☒ Não, retorna um novo array

☐ Depende da condição

☐ Não retorna nada

10 O método `forEach()` retorna um novo array.

☐  Verdadeiro

☒  Falso

11 O método `filter()`

retorna um array com todos os elementos que atendem à condição.

☒ Verdadeiro

☐ Falso

12 Analise o código:

```
let pessoas = [  
  { nome: "Ana", idade: 25 },  
  { nome: "João", idade: 30 },  
  { nome: "Maria", idade: 22 }  
];  
let encontrado = pessoas.find(function(p) {  
  return p.nome === "João";  
});  
console.log(encontrado.idade);
```

Qual será a saída no console?

☐ 25

☒ 30

☐ 22

☐ undefined



Resultado do Quiz

12 / 12

Respostas corretas:

✓ Q1 - Correta

forEach() é usado para percorrer todos os elementos de um array.

✓ Q2 - Correta

map() transforma cada elemento em um novo array com o mesmo tamanho.

✓ Q3 - Correta

filter() filtra elementos que atendem a uma condição.

✓ Q4 - Correta

find() retorna o primeiro elemento que atende à condição.

✓ Q5 - Correta

every() verifica se TODOS os elementos atendem à condição.

✓ Q6 - Correta

map() multiplica cada elemento por 2, resultando em [2, 4, 6, 8].

✓ Q7 - Correta

filter() mantém apenas as idades >= 18, resultando em [18, 22].

✓ Q8 - Correta

reduce() acumula valores em um único resultado.

✓ Q9 - Correta

map() não modifica o array original, retorna um novo array.

✓ Q10 - Correta

forEach() não retorna nada, apenas percorre os elementos.

✓ Q11 - Correta

filter() retorna um novo array com os elementos que passam no teste.

✓ Q12 - Correta

find() encontra João, que tem idade 30, resultando em 30.



PARABÉNS! Você acertou todas! Excelente domínio sobre Métodos de Arrays!



IMPORTANTE: Clique em "Imprimir" para salvar o resultado em PDF.